

Problem Statement- Assignment 2

1. Where would you rate yourself on (LLM, Deep Learning, AI, ML)?

LLM: For LLM I would rate myself as B, as initially I was more focused on python and machine learning but now I have a good understanding about LLM structures and applications, particularly using RAG, Langchain, Langgraph, Embedding models, Vector Databases. I have also built projects using RAG pipeline using Gemini models as APIs. However, I am still learning the internals of pretraining, fine tuning, distributed training, and large scale inference optimization. And for the code understanding, I understand the code well and can make projects under some supervision like DocuChat in my resume where I used RAG pipeline to build a seamless workflow for uploading a document to ask any related question from the document which included indexing, storing data in vector database, embedding and final generation.

Deep Learning: I understand the fundamentals of neural networks, backpropagation, activation functions, CNNs, RNNs, and transformer based architectures at a conceptual level. I have a good theoretical knowledge about various topics and their architectures like CNN, and other machine learning models. I can build and train deep learning models with guidance and understand how modern architectures operate, but I am still gaining experience with advanced optimization techniques and large scale model training. So, I would rate myself as B.

AI: I have a good understanding of core AI concepts and have experience in building AI powered applications. I have good knowledge about machine learning models, workflows, data preprocessing, model evaluation, RAG pipeline structure, APIs, and prompt engineering. While I can build AI applications independently at initial level but would need some supervision for the heavier tasks and workflows and for the deployment stage. Also, I am still exploring the wide range of AI, different libraries and deployment structure so I would rate myself as B.

ML: I am comfortable while building machine learning models. I have deep theoretical as well as practical knowledge of building machine learning models and its different algorithms like supervised (SVM, Logistic Regressions) and unsupervised (K means). I am proficient in python, pandas, numpy, scikit-learn, matplotlib and other workflows. I have worked on different projects like Image Classification, Fake News Detections and other visualization projects. And for python I have independently completed a core python automation project in my last internship. So, I would rate myself as A in Machine Learning.

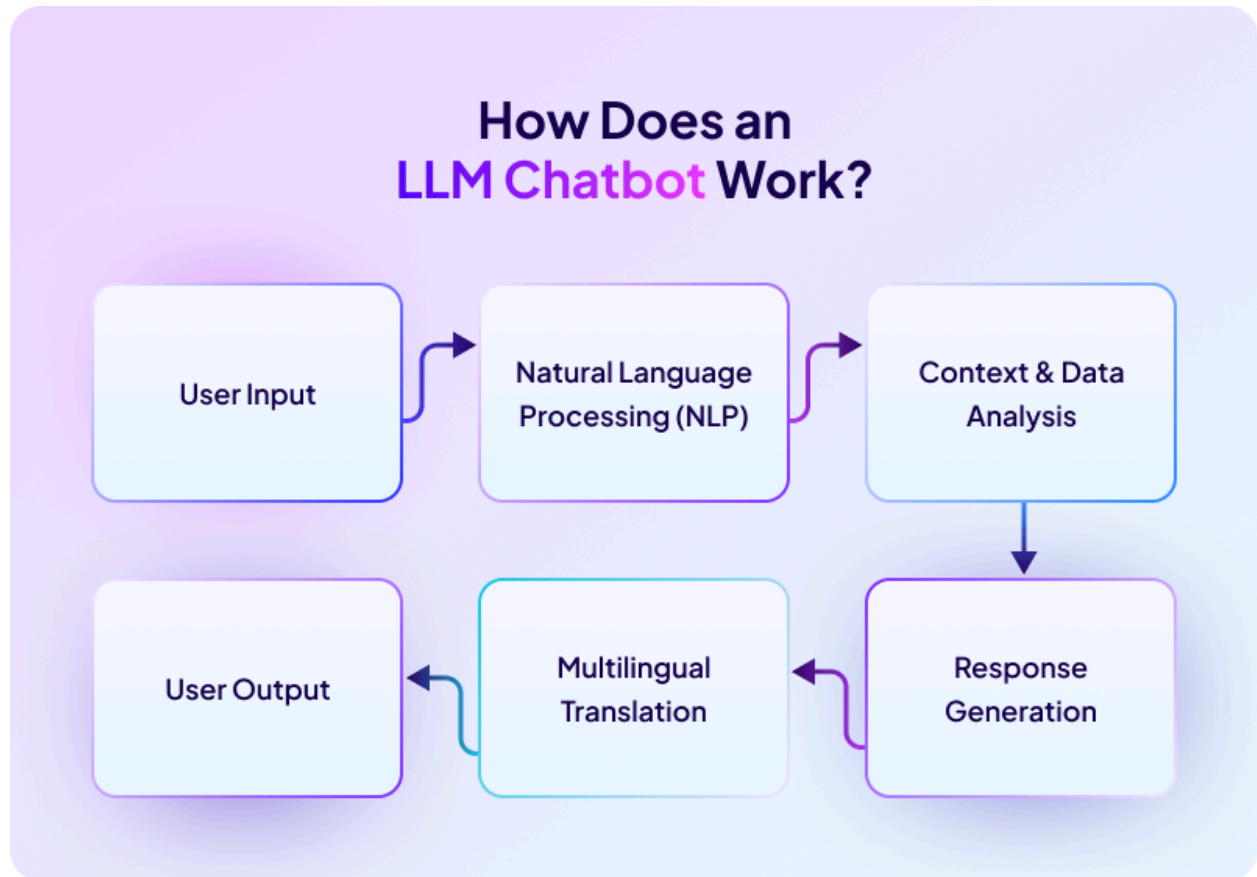
2. What are the key architectural components to create a chatbot based on an LLM? Please explain the approach on a high-level.

A LLM chatbot is an AI system capable of understanding user queries and process it to produce a relevant outcome that is human like and human understandable. Chatbots use APIs, relevant models, embedding models and databases to deeply understand the query, store appropriate information for a longer period and generate most appropriate responses.

Key architectural components to create a chatbot:

- The **User Interface Layer** through which user interacts and queries are passed. Interface layer may include Streamlit interface, Web Applications, Mobile Applications or any other internet portal. Here, prompt engineering is done to put the prompt accurately to get the closest answer. This also includes user validation, prompt validation and other security checks before the prompt is passed.
- Next comes the **LLM Layer** which becomes the brain of the workflow where all the queries are processed, context is understood and responses are generated in natural, human understandable language. For example: GPT models, Gemini models etc.
- **RAG Pipeline** which combines all these steps from query generation to indexing, then converting it into vector embeddings, storing it is into vector stores or databases and applying various semantic or other search operations to find the most appropriate answer.
- **Embedding Model** is used to overcome knowledge limitation by converting and information resource into numerical vector representation which makes it easy to find the nearest output to the query using embeddings. Embeddings capture semantic meaning and allow the system to find relevant information even when exact keywords do not match.
- **Vector Databases** like FAISS, PineCone are used to store the vector embeddings from where they can be used for semantic search between the user query and the relevant answer.
- **Memory Component** is used to store memory of various user queries and their information provided earlier which can be useful in multi turn conversations or for personalized responses. This is used by LLM to understand the context of present query by remembering past data or information provided.

- The last step includes the **Response Generation** where user query is combined with the relevant documents and the conversation history to generate the closest response in human understandable language.



3. Please explain vector databases. If you were to select a vector database for a hypothetical problem, which one will you choose and why?

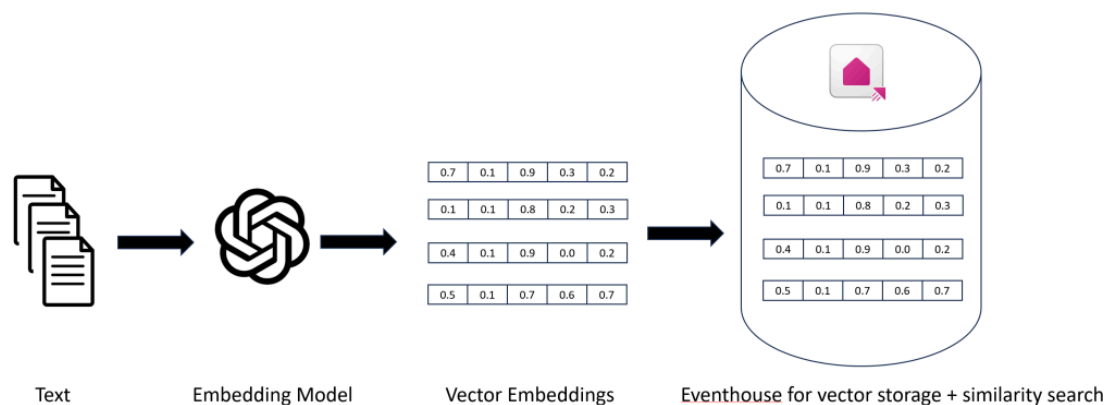
A vector database is a specialized database designed to store and search vector embeddings efficiently. It is different from a vector store in terms of storage and configuration of high level data.

Earlier data was fetched from databases using sql queries like
`SELECT * FROM books WHERE title='Python'`

To fetch the data of all the books in the database which are titled as Python.

But Vector Databases serve different purpose; they store high dimensional vectors which is the data converted into numerics for better search purposes.

For example:



As a result, vector databases can perform similarity search rather than exact keyword matching.

Vector Databases are important because LLMs do not directly know the user specific or any other related information. For example a company has documents about HR policies, internal documentation or any other knowledge bases which are not directly used for retrieval but first they are converted into vectors and stored as embeddings in these databases. When users ask questions, the most relevant documents are retrieved and supplied to the LLM. This is the foundation of Retrieval-Augmented Generation (RAG).

Important Components of Vector Database are:

- Embeddings- where the text from any document or information source is converted into numerical vectors using embedding model like Open AI Embedding.
- Indexing- it allows fast retrieval of documents from large amount of vectors stored in database.
- Similarity Search- search techniques like cosine similarity, Euclidean distance are used to calculate the similarity between the user query and the information stored to retrieve the most relevant output from all the data stored in the database.

Hypothetical Problem and Database Selection:

Let's take a situation where we have to make a AI powered document uploader Student Portal for a college or school where students can upload various documents about "Student Regulations", "Course handbooks", "Placement guidelines" etc. and students can ask any relatable question like "What is the minimum attendance requirement?" or "How many credits are needed for graduation?"

Then this portal should be feasible in terms of memory storage, and its accessibility on any internet platform.

For solving this purpose I will choose FAISS(Facebook AI Similarity Search) for serving various purposes like:

- It is an open source and free platform for its seamless usage by every student. Also, it does not need any high cloud infrastructure for the storage.
- Its high performance on similarity search which will help to generate quick and relevant responses on the portal.
- Also, it seamlessly integrates with various frameworks like Langchain, and Gemini Embeddings which helps to easily update or make any change in the portal at any minor issue caused.

FAISS due to its speed, simplicity, open source nature, and seamless integration with modern LLM frameworks, it provides an efficient and practical solution for building accurate, context aware AI assistants.